

Questionário I

Pergunta 1

0,25 em 0,25 pontos



Considere a seguinte configuração de uma aplicação ASP.NET Core:

```
app.UseHttpsRedirection();  
app.UseDefaultFiles();  
app.UseStaticFiles();  
  
app.MapGet("/api/pets", () =>  
    Results.Ok(new[] { "Mingau",  
        "Luna" }));  
  
app.Run();
```

O diretório wwwroot contém apenas o arquivo index.html e não contém arquivo correspondente a /api/pets. Considere que as requisições já chegam por HTTPS, de modo que UseHttpsRedirection() não produz redirecionamento. Qual alternativa descreve corretamente o tratamento de / e /api/pets?

Respost
a
Sele
ciona
da:

✓ d.

UseDefaultFiles() reescreve a requisição à raiz para o arquivo padrão; UseStaticFiles() serve esse arquivo; e /api/pets continua sendo atendido pelo endpoint mapeado.

Respost
as:

a.

UseDefaultFiles() envia diretamente o conteúdo de index.html, enquanto UseStaticFiles() encaminha /api/pets ao endpoint mapeado.

b.

A requisição à raiz somente será atendida depois que o roteamento verificar se existe um endpoint correspondente a /index.html.

c.

UseStaticFiles() procura automaticamente index.html ao receber /, tornando UseDefaultFiles() dispensável nessa configuração.

✓ d.

UseDefaultFiles() reescreve a requisição à raiz para o arquivo padrão; UseStaticFiles() serve esse arquivo; e /api/pets continua sendo atendido pelo endpoint mapeado.

e.

O middleware de arquivos estáticos atende apenas arquivos CSS, JavaScript e imagens; por isso, index.html precisaria ser exposto por MapGet().

Comentário da resposta:

Resposta: D

Comentário: UseDefaultFiles() não envia o arquivo; ele reescreve / para /index.html quando encontra o documento padrão. UseStaticFiles() serve index.html. Como não existe arquivo estático correspondente a /api/pets, essa requisição segue no pipeline e é atendida pelo endpoint mapeado.

Pergunta 2

0,25 em 0,25 pontos



Uma API criada com Minimal APIs possui os seguintes endpoints:

```
GET /api/pets/{id:int}
```

```
GET  
/api/pets?especie=Gato&cidade=  
Curitiba
```

```
POST /api/interesses
```

O endpoint POST /api/interesses recebe, no corpo da requisição, um objeto JSON com PetId, Nome, Email e Mensagem.

De acordo com as convenções de vinculação de parâmetros das Minimal APIs, qual alternativa está correta?

Resposta
Selec
ionad
a:



a.

id é obtido da rota; especie e cidade são obtidos da query string; e o objeto enviado ao POST pode ser desserializado do corpo JSON.

Resposta
s:



a.

id é obtido da rota; especie e cidade são obtidos da query string; e o objeto enviado ao POST pode ser desserializado do corpo JSON.

b.

id é obtido da query string; especie e cidade são obtidos da rota; e o objeto enviado ao POST é desserializado do corpo JSON.

c.

id é obtido da rota; especie e cidade são obtidos do corpo da requisição; e o objeto enviado ao POST é obtido da query string.

d.

id é obtido da rota; especie e cidade são obtidos dos cabeçalhos; e o objeto enviado ao POST é obtido da query string.

e.

id, especie e cidade são obtidos da query string; e o objeto enviado ao POST precisa ser desserializado manualmente.

Comentário da resposta:

Resposta: A

Comentário: Um parâmetro presente no template é associado à rota. Parâmetros simples não presentes nela podem ser obtidos da query string, enquanto um objeto complexo pode ser associado ao corpo JSON.

Pergunta 3

0,25 em 0,25 pontos



Uma API mantém a seguinte coleção de animais:

Mingau	Gato	Pequeno	São Paulo
Amora	Gato	Pequeno	Florianópolis
Luna	Cachorro	Médio	Curitiba
Thor	Cachorro	Grande	Belo Horizonte

Os filtros informados na requisição são combinados. Para espécie e porte, a comparação ignora diferenças entre letras maiúsculas e minúsculas. Para cidade, a API verifica se o texto informado está contido no nome da cidade, também sem diferenciar maiúsculas de minúsculas.

Considere a requisição:

```
GET
/api/pets?especie=gato&porte=PEQUENO&cidade=paulo
```

Qual resultado será retornado?

Respost
a
Sele
cion
ada:



e.

Apenas Mingau, porque ele atende simultaneamente aos filtros de especie, porte e cidade.

Respost
as:

a.

Mingau e Amora, porque ambos atendem aos filtros de especie e porte.

b.

Mingau, Amora e Thor, porque o filtro textual de cidade é aplicado independentemente dos demais critérios.

c.

Apenas Amora, porque o filtro de cidade é aplicado somente quando especie e porte são diferentes.

d.

Nenhum animal, porque o texto informado para especie e porte não possui a mesma combinação de maiúsculas e minúsculas dos dados armazenados.



e.

Apenas Mingau, porque ele atende simultaneamente aos filtros de especie, porte e cidade.

Comentário da resposta:

Resposta: E

Comentário: Os filtros são cumulativos. Mingau atende aos três critérios. Amora satisfaz espécie e porte, mas Florianópolis não contém o texto paulo.

Pergunta 4

0,25 em 0,25 pontos



Um endpoint executa as validações na seguinte ordem:

```
if (request.PetId <= 0 ||  
    string.IsNullOrEmpty(request.Nome) ||  
    string.IsNullOrEmpty(request.Email) ||  
    string.IsNullOrEmpty(request.Mensagem) ||  
    !request.Email.Contains('@'))  
{  
    return Results.BadRequest();  
}  
  
var pet = pets.FirstOrDefault(p => p.Id ==  
    request.PetId);  
  
if (pet is null)  
{  
    return Results.NotFound();  
}  
  
return Results.Ok();
```

Considere duas requisições:

- I. PetId = 999, Nome = "Ana",
Email = "ana.exemplo.com", Mensagem = "Tenho
interesse"
- II. PetId = 999, Nome = "Ana",
Email = "ana@exemplo.com", Mensagem = "Tenho
interesse"

O animal de identificador 999 não existe. Quais respostas são produzidas?

Resposta
Seleci
onada:



b.

I retorna 400; II retorna 404.

Respostas
:

a.

I retorna 404; II retorna 400.



b.

I retorna 400; II retorna 404.

c.

Ambas retornam 404, pois a
inexistência do animal prevalece
sobre a validação.

d.

Ambas retornam 400, pois um
identificador inexistente é tratado
como entrada inválida.

e.

I retorna 400; II retorna 200, pois os
dados da segunda requisição são
formalmente válidos.

Comen
tári
o
da
res
pos
ta:

Resposta: B

Comentário: A primeira requisição é encerrada durante a validação porque o e-mail não contém @. A segunda supera essa etapa, mas a busca pelo animal não encontra o identificador informado.

Pergunta 5

0,25 em 0,25 pontos



Uma aplicação registra seus endpoints por meio de um método de extensão:

```
app.UseHttpsRedirection();  
  
app.UseDefaultFiles();  
app.UseStaticFiles();  
  
app.MapPetEndpoints();  
  
app.Run();
```

O método é definido assim:

```
public static WebApplication  
    MapPetEndpoints(  
        this WebApplication app)  
{  
    var api = app.MapGroup("/api");  
  
    api.MapGet("/pets", ...);  
    api.MapGet("/pets/{id:int}", ...);  
    api.MapGet("/boas-vindas/{nome}",  
        ...);  
    api.MapPost("/interesses", ...);  
  
    return app;  
}
```

Qual efeito essa organização produz?

Resposta
Selec
ionad
a:

✓ c.

Os endpoints continuam sendo Minimal APIs, e o grupo acrescenta /api aos caminhos registrados nele.

Resposta
s:

a.

O método de extensão registra os endpoints, mas MapGroup("/api") afeta apenas a organização do código e não as URLs.

b.

Os endpoints passam a ser controladores MVC porque foram retirados de Program.cs.

✓ c.

Os endpoints continuam sendo Minimal APIs, e o grupo acrescenta /api aos caminhos registrados nele.

d.

O método de extensão cria uma segunda instância de WebApplication, responsável exclusivamente pelos endpoints agrupados.

e.

O agrupamento mantém as URLs anteriores e adiciona versões alternativas com o prefixo /api.

Comen
tári
o
da
res
pos
ta:

Resposta: C

Comentário: A refatoração modifica a organização do código, não o modelo usado para definir os endpoints. MapGroup("/api") estabelece um prefixo comum para as rotas registradas no grupo.

Pergunta 6

0,25 em 0,25 pontos



Uma aplicação ASP.NET Core MVC contém:

```
builder.Services.AddControllersWithViews();

var app = builder.Build();

app.UseHttpsRedirection();
app.UseStaticFiles();

app.MapControllerRoute(
    name: "default",
    pattern:
        "{controller}/{action=Index}/{id?}");

app.Run();
```

Existe um `ProdutosController` com uma ação pública `Index()`. O que ocorre ao acessar `/Produtos`?

Resposta
Selec
ionad
a:



e.

O primeiro segmento identifica
ProdutosController, e a ausência do
segundo faz a rota utilizar a ação
padrão Index.

Resposta
s:

a.

Produtos é interpretado como nome da
ação, enquanto o controlador recebe o
valor padrão Index.

b.

A URL não corresponde à rota, pois id
precisa ser informado quando o
segmento da ação é omitido.

c.

O middleware de arquivos estáticos
procura Produtos.html e transforma a
requisição em uma chamada MVC
caso o arquivo não exista.

d.

AddControllersWithViews() cria
automaticamente uma rota exclusiva
para cada controlador, tornando
MapControllerRoute() desnecessário.



e.

O primeiro segmento identifica
ProdutosController, e a ausência do
segundo faz a rota utilizar a ação
padrão Index.

Comentário da resposta:

Resposta: E

Comentário: No roteamento convencional, o primeiro segmento representa o controlador. Como a ação possui o valor padrão Index e id é opcional, /Produtos pode ser associado a `ProdutosController.Index()`.

Pergunta 7

0,25 em 0,25 pontos



Um ViewModel é definido assim:

```
public class ProdutoViewModel
{
    [Required]
    [StringLength(60, MinimumLength = 2)]
    public string Nome { get; set; } = "";

    [Range(0, 100)]
    public int Quantidade { get; set; }

    [Required]
    [StringLength(200, MinimumLength = 20)]
    public string Descricao { get; set; } = "";
}
```

Uma ação POST verifica ModelState.IsValid e retorna a mesma view quando o estado é inválido. O formulário envia:

```
Nome = "A"

Quantidade = 15

Descricao = "Descrição curta"
```

Qual comportamento é esperado?

Respost
a
Sele
cionada:

✓ a.

Nome e Descricao violam seus
comprimentos mínimos, tornando
ModelState inválido.

Respost
as:

✓ a.

Nome e Descricao violam seus
comprimentos mínimos, tornando
ModelState inválido.

b.

Somente Nome é inválido, pois a
primeira falha impede a avaliação
das demais propriedades.

c.

O modelo é válido, pois todas as
propriedades obrigatórias foram
preenchidas e a quantidade está no
intervalo permitido.

d.

A requisição é rejeitada pelo roteamento
antes da execução da ação porque
Data Annotations participam da
seleção de endpoints.

e.

O model binder completa
automaticamente as strings até os
comprimentos mínimos definidos.

Comen
tári
o
da
res
po
sta
:

Resposta: A

Comentário: Preencher uma propriedade obrigatória não garante que ela satisfaça outras restrições. Nome e Descrição estão abaixo dos comprimentos mínimos, enquanto Quantidade está dentro do intervalo permitido.

Pergunta 8

0,25 em 0,25 pontos



Um controlador MVC possui:

```
[HttpGet]

public IActionResult Create()

{

    return View(new
        ProdutoViewModel());

}


[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult
Create(ProdutoViewModel model)
{
    if (!ModelState.IsValid)
        return View(model);

    Salvar(model);
    TempData["MensagemSucesso"]
= "Produto cadastrado.";
    return
    RedirectToAction(nameof(Index));
}
```

Qual alternativa descreve corretamente as responsabilidades desse fluxo?

Resposta
Selec
ionad
a:

 d.

O POST recebe os dados associados ao ViewModel, verifica sua validação e, se forem válidos, executa a operação e retorna um redirecionamento.

Resposta
s:

a.

A ação GET recebe os dados enviados pelo formulário, enquanto o POST apenas escolhe a próxima view.

b.

TempData precisa ser preenchido antes da validação porque seus valores não permanecem disponíveis após redirecionamentos.

c.

[ValidateAntiForgeryToken] executa as regras de [Required], [Range] e [StringLength], tornando ModelState desnecessário.

 d.

O POST recebe os dados associados ao ViewModel, verifica sua validação e, se forem válidos, executa a operação e retorna um redirecionamento.

e.

RedirectToAction() executa a ação Index dentro da mesma resposta HTTP, sem nova requisição do cliente.

Comentário da resposta:

Resposta: D

Comentário: O GET prepara a exibição do formulário. O POST recebe os dados, verifica o estado de validação e executa a operação somente quando o modelo é válido.

Pergunta 9

0,25 em 0,25 pontos



Uma aplicação registra um filtro MVC com o seguinte código:

```
public void
    OnActionExecuting(ActionExecutingContext context)
{
    if (context.Controller is Controller controller)
    {
        controller.ViewData["Contexto"] =
            "Área administrativa";
    }
}
```

A mesma informação deve estar disponível nas views de diversas ações. Qual justificativa é mais adequada para o uso desse filtro?

Resposta
Seleci
onada
:

✔ b.

Centralizar uma preocupação transversal executada antes das ações, evitando repetir o mesmo código em diferentes controladores.

Resposta
s:

a.

Concentrar no filtro as regras de negócio utilizadas pelas ações, reduzindo a quantidade de serviços da aplicação.

✔ b.

Centralizar uma preocupação transversal executada antes das ações, evitando repetir o mesmo código em diferentes controladores.

c.

Utilizar o filtro como mecanismo principal de roteamento para decidir qual ação será executada.

d.

Manter o conteúdo de ViewData disponível globalmente para todas as requisições futuras da aplicação.

e.

Substituir a validação dos modelos recebidos pelas ações, pois o filtro é executado antes delas.

Comentário da resposta:

Resposta: B

Comentário: Filtros são adequados a comportamentos transversais do pipeline MVC. Nesse caso, o filtro prepara uma informação comum sem duplicar a mesma instrução em diferentes controladores.

Pergunta 10

0,25 em 0,25 pontos



Uma aplicação MVC possui:

```
Views/  
├─ _ViewImports.cshtml  
├─ Shared/  
│   ├─ _Layout.cshtml  
│   └─ _ItemCard.cshtml  
└─ Produtos/  
    ├─ Index.cshtml  
    └─ Create.cshtml
```

Em Index.cshtml:

```
@foreach (var item in Model)  
{  
    <partial name="_ItemCard"  
        model="item" />  
}
```

Em _Layout.cshtml:

```
<main>  
    @RenderBody()  
</main>
```

Qual alternativa caracteriza corretamente esses elementos?

Respost
a
Sele
cion
ada:

✔ c.

O layout define a estrutura compartilhada; a partial encapsula um fragmento reutilizável; e `RenderBody()` indica onde o conteúdo da view será renderizado.

Respost
as:

a.

O layout executa o controlador da página, a partial substitui a ação e `RenderBody()` realiza o model binding.

b.

O layout é destinado apenas a referências CSS; a partial cria rotas MVC; e `RenderBody()` seleciona o controlador.

✔ c.

O layout define a estrutura compartilhada; a partial encapsula um fragmento reutilizável; e `RenderBody()` indica onde o conteúdo da view será renderizado.

d.

Layout e partial representam o mesmo mecanismo, mas partials não podem receber modelos tipados.

e.

A partial concentra regras de negócio reutilizáveis, enquanto o layout executa regras de validação do `ViewModel`.

Comentário da resposta:

Resposta: C

Comentário: layout e partial promovem reutilização em níveis diferentes. O layout fornece a estrutura comum das páginas, enquanto a partial reutiliza fragmentos de interface e pode receber um modelo.

Questionário II

Pergunta 1

0,25 em 0,25 pontos



Uma aplicação inicialmente armazena produtos em uma `List<Produto>` criada durante a inicialização do processo. Posteriormente, passa a utilizar SQLite com Entity Framework Core e um arquivo `dados.db`.

Qual mudança é mais relevante do ponto de vista da persistência?

Resposta
Seleci
onada
:

✓ a.

Os registros deixam de depender exclusivamente da duração do processo e podem ser recuperados do banco em execuções posteriores.

Resposta
s:

✓ a.

Os registros deixam de depender exclusivamente da duração do processo e podem ser recuperados do banco em execuções posteriores.

b.

Os objetos deixam de existir em memória e passam a ser manipulados diretamente no arquivo pelo código C#.

c.

O SQLite preserva os mesmos objetos C# em memória depois que o processo é encerrado.

d.

O banco elimina a necessidade de representar registros por objetos durante a execução da aplicação.

e.

Qualquer alteração feita em uma entidade é gravada imediatamente no SQLite, independentemente de SaveChanges().

Comentário da resposta:

Resposta: A

Comentário: Uma coleção mantida apenas em memória depende do ciclo de vida do processo. Um banco em arquivo permite que os registros permaneçam disponíveis após o encerramento da aplicação.

Pergunta 2

0,25 em 0,25 pontos



Considere a configuração:

```
builder.Services.AddDbContextPool<AppDbContext>(options =>
    options.UseSqlite(
        "Data Source=dados.db;Pooling=True"));
```

Qual alternativa diferencia corretamente os dois mecanismos de pooling presentes nesse cenário?

Respost
a
Sele
cion
ada:

✔ d.

O parâmetro da conexão atua no pooling de conexões do provedor, enquanto `AddDbContextPool()` permite reutilizar instâncias de `DbContext` depois da reinicialização de seu estado.

Respost
as:

a.

`Pooling=True` reutiliza instâncias de `DbContext`, enquanto `AddDbContextPool()` reutiliza conexões com o SQLite.

b.

Ambos configuram o mesmo pool, e utilizá-los juntos apenas duplica sua capacidade.

c.

`AddDbContextPool()` compartilha entidades consultadas entre requisições, enquanto `Pooling=True` mantém resultados SQL em cache.

✔ d.

O parâmetro da conexão atua no pooling de conexões do provedor, enquanto `AddDbContextPool()` permite reutilizar instâncias de `DbContext` depois da reinicialização de seu estado.

e.

`Pooling=True` preserva transações concluídas, enquanto `AddDbContextPool()` mantém migrations em memória.

Comentário da resposta:

Resposta: D

Comentário: pooling de contexto e pooling de conexão atuam em níveis diferentes. Um reutiliza instâncias de DbContext; o outro atua sobre as conexões mantidas pelo provedor.

Pergunta 3

0,25 em 0,25 pontos



Uma entidade utiliza o seguinte mapeamento:

```
public int Id { get; set; }

public string CategoriasTexto { get; set;
    } = "";

[NotMapped]
public string[] Categorias
{
    get => CategoriasTexto.Split(
        '|',

        StringSplitOptions.RemoveEmptyEntries |

        StringSplitOptions.TrimEntries);

    set => CategoriasTexto =
        string.Join('|', value);
}
```

Qual interpretação está correta?

Resposta
Seleci
onada
:

✔ b.

CategoriasTexto é persistida, enquanto Categorias oferece uma representação conveniente em memória sem possuir coluna própria.

Resposta
s:

a.

Categorias e CategoriasTexto são persistidas como duas colunas, mantidas automaticamente em sincronismo pelo EF Core.

✔ b.

CategoriasTexto é persistida, enquanto Categorias oferece uma representação conveniente em memória sem possuir coluna própria.

c.

Categorias representa uma relação persistente com outra tabela, enquanto CategoriasTexto existe apenas para a interface.

d.

[NotMapped] faz com que toda a entidade deixe de participar do rastreamento de alterações.

e.

[NotMapped] permite que a propriedade seja gravada em inserções, mas impede sua leitura em consultas.

Comen
tári
o
da
res
pos
ta:

Resposta: B

Comentário: [NotMapped] exclui
Categorias do mapeamento
persistente. O conteúdo armazenado
permanece em CategoriasTexto, e a
propriedade não mapeada oferece
outra representação desses dados em
memória.

Pergunta 4

0,25 em 0,25 pontos



Uma aplicação que utiliza SQLite adota concorrência otimista com um token gerenciado pela própria aplicação. A entidade possui:

```
public Guid Version { get; set; } =  
    Guid.NewGuid();
```

O mapeamento contém:

```
builder.Property(x => x.Version)  
    .IsConcurrencyToken();
```

A aplicação atribui um novo Guid a Version antes de cada SaveChangesAsync() e usa como valor original o token recebido pelo cliente. Dois usuários, A e B, carregam a mesma versão de um registro. A salva primeiro, fazendo o token mudar. Em seguida, B tenta salvar com o token original recebido ao abrir o registro.

O que pode ocorrer na tentativa de B?

Resposta
Selec
ionad
a:



e.
A atualização de B pode não afetar nenhuma linha e SaveChangesAsync() pode lançar DbUpdateConcurrencyException.

Resposta
s:

a.

A alteração de B necessariamente substitui a de A, porque a última gravação sempre prevalece.

b.

A alteração de A é desfeita automaticamente quando B envia um token de versão anterior.

c.

O EF Core bloqueia o registro desde a consulta inicial, impedindo que A e B o editem simultaneamente.

d.

O EF Core identifica as propriedades modificadas por cada usuário e combina automaticamente as duas alterações.



e.
A atualização de B pode não afetar nenhuma linha e SaveChangesAsync() pode lançar DbUpdateConcurrencyException.

Comentário:

Resposta: E

Comentário: O EF Core inclui o valor original do token de concorrência na condição da atualização. Como A mudou o token, a operação de B não encontra a linha esperada; zero linhas afetadas leva a DbUpdateConcurrencyException. No SQLite, o token não é atualizado automaticamente como um rowversion do SQL Server, por isso o enunciado explicita a atualização pela aplicação.

Pergunta 5

0,25 em 0,25 pontos



Considere a consulta:

```
var consulta = dbContext.Produtos
    .AsNoTracking()
    .Where(p => p.Ativo);

if
    (!string.IsNullOrEmpty(catego
        ria))
{
    consulta = consulta.Where(
        p => p.Categoria ==
        categoria);
}

var produtos = await consulta
    .OrderBy(p => p.Nome)
    .Select(p => new
        ProdutoResponse(
            p.Id,
            p.Nome,
            p.Categoria,
            p.Preco))
    .ToListAsync();
```

Qual análise está correta?

Respost
a
Sele
ciona
da:

 c.

A consulta continua sendo composta até a materialização; AsNoTracking() evita o rastreamento das entidades nesse cenário de leitura; e Select() define a projeção do resultado.

Respost
as:

a.

AsNoTracking() impede futuras atualizações da tabela, e os filtros posteriores são aplicados somente depois que os registros são carregados.

b.

Select() exige primeiro materializar entidades completas e somente depois descarta as propriedades não utilizadas.

 c.

A consulta continua sendo composta até a materialização; AsNoTracking() evita o rastreamento das entidades nesse cenário de leitura; e Select() define a projeção do resultado.

d.

Em razão da execução diferida do LINQ, ToListAsync() ocorre antes de Where() e OrderBy().

e.

`AsNoTracking()` cria um cache local dos registros e `Select()` transforma a operação em uma transação somente leitura.

Comen
tári
o
da
res
pos
ta:

Resposta: C

Comentário: Os filtros, a ordenação e a projeção são compostos antes da materialização realizada por `ToListAsync()`. `AsNoTracking()` é apropriado quando as entidades consultadas não precisam ser rastreadas para atualização.

Pergunta 6

0,25 em 0,25 pontos



Uma entidade de persistência contém propriedades internas utilizadas pelo Entity Framework Core, inclusive um token de concorrência. O endpoint responde com:

```
public sealed record  
    ProdutoResponse(  
  
        int Id,  
  
        string Nome,  
  
        string Categoria,  
  
        decimal Preco,  
  
        bool Disponivel);
```

O objeto retornado é criado por projeção a partir da entidade. Qual é a principal vantagem dessa decisão?

Resposta
Selecionada
:



d.

O contrato HTTP fica separado do modelo de persistência, permitindo definir explicitamente quais dados são expostos ao cliente.

Respostas:

a.

O DTO impede que propriedades omitidas sejam consultadas pelo EF Core em qualquer outra operação.

b.

A criação do DTO elimina a entidade do modelo de persistência depois que a consulta é executada.

c.

O DTO cria automaticamente novas versões da API sempre que a entidade sofre uma alteração.



d.

O contrato HTTP fica separado do modelo de persistência, permitindo definir explicitamente quais dados são expostos ao cliente.

e.

A utilização de DTO torna desnecessário documentar os schemas e códigos de resposta da API.

Comentário da resposta:

Resposta: D

Comentário: Separar a representação externa da entidade reduz o acoplamento entre o banco e o contrato HTTP. Propriedades necessárias à persistência não precisam ser expostas ao cliente.

Pergunta 7

0,25 em 0,25 pontos



Uma API recebe os parâmetros:

```
public class ProdutoQuery
{
    private const int MaxPageSize = 20;

    public int Page { get; set; } = 1;

    private int _pageSize = 6;

    public int PageSize
    {
        get => _pageSize;
        set => _pageSize =
            Math.Clamp(value, 1,
                MaxPageSize);
    }
}
```

A paginação é aplicada assim:

```
var page = Math.Max(query.Page, 1);

var totalItems = await
    consulta.CountAsync();

var items = await consulta
    .Skip((page - 1) * query.PageSize)
    .Take(query.PageSize)
    .ToListAsync();
```

Há 47 registros. O cliente envia:

?page=3&pageSize=50

Qual resultado decorre dessa lógica?

Respost
a
Sele
cion
ada:



b.

A página efetiva é 3, o tamanho é limitado a 20, são ignorados 40 registros e podem ser retornados no máximo 7 itens.

Respost
as:

a.

A página efetiva é 3 e o tamanho permanece 50, permitindo que os 47 registros sejam devolvidos.



b.

A página efetiva é 3, o tamanho é limitado a 20, são ignorados 40 registros e podem ser retornados no máximo 7 itens.

c.

Como pageSize ultrapassa o máximo, page e pageSize retornam aos valores padrão.

d.

O model binding responde necessariamente com 400 Bad Request, pois Math.Clamp() rejeita valores superiores ao máximo.

e.

Os valores são invertidos, fazendo a terceira página conter 20 páginas com 3 registros cada.

Comentário da resposta:

Resposta: B

Comentário: O setter limita PageSize a 20. Na terceira página, `Skip((3 - 1) * 20)` ignora 40 registros, restando apenas 7 dos 47 existentes.

Pergunta 8

0,25 em 0,25 pontos



Um endpoint retorna uma resposta estruturada quando não encontra determinado produto:

```
return produto is null  
    ? Results.Problem(  
        title: "Produto não encontrado.",  
        detail: $"Não existe produto com  
        id {id}.",  
        statusCode:  
        StatusCodes.Status404NotFound)  
    : Results.Ok(produto);
```

A aplicação também utiliza os serviços de ProblemDetails.

Qual é a vantagem de representar a falha dessa maneira?

Resposta
Seleci
onada
:



e.

Manter o código HTTP 404 e fornecer ao cliente uma representação estruturada com informações sobre o problema.

Resposta
s:

a.

Transferir ao cliente a responsabilidade de definir qual código HTTP corresponde ao problema informado.

b.

Converter a resposta 404 em 200, permitindo que o cliente determine o erro somente pelo conteúdo recebido.

c.

Utilizar uma única resposta para qualquer falha, sem necessidade de distinguir erros como 400, 404 ou 409.

d.

Fazer com que o roteamento tente localizar automaticamente outro produto quando o identificador não existir.



e.

Manter o código HTTP 404 e fornecer ao cliente uma representação estruturada com informações sobre o problema.

Comen
tári
o
da
res
pos
ta:

Resposta: E

Comentário: O código de status e o corpo da resposta cumprem funções complementares. O status mantém a semântica HTTP da falha, enquanto ProblemDetails fornece uma estrutura previsível para descrever o problema.

Pergunta 9

0,25 em 0,25 pontos



Uma equipe utiliza OpenAPI, Swagger UI e NSwag em uma API ASP.NET Core. Qual alternativa descreve adequadamente a relação entre essas tecnologias?

Resposta
Selecionada:
a:

✓ a.

OpenAPI descreve formalmente o contrato da API; Swagger UI pode apresentar esse contrato de forma interativa; e NSwag pode consumir um documento OpenAPI para gerar código cliente.

Respostas:

✓ a.

OpenAPI descreve formalmente o contrato da API; Swagger UI pode apresentar esse contrato de forma interativa; e NSwag pode consumir um documento OpenAPI para gerar código cliente.

b.

Swagger UI e NSwag são equivalentes porque ambos precisam apresentar a mesma interface gráfica ao usuário.

c.

OpenAPI é uma interface gráfica; Swagger UI gera exclusivamente clientes TypeScript; e NSwag define a especificação HTTP utilizada pela API.

d.

Swagger define o contrato durante o desenvolvimento, enquanto OpenAPI existe apenas para geração de clientes em produção.

e.

Depois que o NSwag gera o cliente, a Swagger UI passa a inferir os endpoints diretamente pelo código TypeScript.

Comentário da resposta:

Resposta: A

Comentário: OpenAPI descreve o contrato da API. Swagger UI pode apresentar esse documento de forma interativa, enquanto NSwag pode utilizá-lo como entrada para gerar clientes.

Pergunta 10

0,25 em 0,25 pontos



Um projeto ASP.NET Core direcionado ao .NET 10 referencia os pacotes Microsoft.Extensions.ApiDescription.Server e NSwag.MSBuild. O arquivo .csproj contém:

```
<PropertyGroup>

  <OpenApiGenerateDocuments>true
</OpenApiGenerateDocuments>

  <OpenApiDocumentsDirectory>

    $(MSBuildProjectDirectory)/obj/openapi

  </OpenApiDocumentsDirectory>

  <OpenApiGenerateDocumentsOptions>
    --file-name v1
  </OpenApiGenerateDocumentsOptions>
</PropertyGroup>

<Target
  Name="GenerateTypeScriptClient"

  AfterTargets="GenerateOpenApiDocuments">
    <Error

      Condition="!Exists('$(OpenApiDocumentsDirectory)/v1.json')"
      Text="Documento OpenAPI não encontrado." />

    <Exec
      WorkingDirectory="$(MSBuildProjectDirectory)"

      Command="$(NSwagExe_Net100)
run nswag.json" />
    </Exec>
  </Target>
```

O arquivo nswag.json usa como entrada:

```
obj/openapi/v1.json
```

e gera:

```
Clients/TypeScript/api-client.ts
```

Considerando que os dois pacotes estão instalados em versões compatíveis com o projeto, qual sequência é coerente com essa configuração?

Respost
a
Sele
cion
ada:



c.

O build gera primeiro v1.json no diretório configurado; depois o target executa o NSwag para gerar o cliente TypeScript a partir desse documento.

Respost
as:

a.

O NSwag cria v1.json; depois a Swagger UI transforma o arquivo em TypeScript.

b.

A Swagger UI precisa permanecer aberta durante o build para fornecer o documento ao NSwag.



c.

O build gera primeiro v1.json no diretório configurado; depois o target executa o NSwag para gerar o cliente TypeScript a partir desse documento.

d.

O cliente TypeScript é criado antes do documento e passa a ser usado para reconstruir v1.json.

e.

A geração do OpenAPI e a do cliente são independentes, portanto a existência de v1.json não interfere no target.

Com
e
nt
ár
io
d
a
re
s
p
o
st
a:

Resposta: C

Comentário:

Microsoft.Extensions.ApiDescription
.Server gera o documento OpenAPI
no build. A opção --file-name v1
torna explícito o nome esperado, e
AfterTargets executa o target
personalizado depois da geração.
Em um projeto .NET 10,
NSwag.MSBuild disponibiliza
NSwagExe_Net100 para executar
nswag.json e produzir o cliente
TypeScript.

Questionário III

Pergunta 1

0,25 em 0,25 pontos

Durante o desenvolvimento, uma API ASP.NET Core executa em:

`http://localhost:5213`

e o frontend React em:

`http://localhost:5173`

A API registra:

```
builder.Services.AddCors(options =>
{
    options.AddPolicy("FrontendDev", policy =>
    {
        policy

        .WithOrigins("http://localhost:5173")
        .AllowAnyHeader()
        .AllowAnyMethod();
    });
});
```

Após `app.Build()`, o pipeline executa `app.UseCors("FrontendDev")` antes dos endpoints. Qual é o papel dessa política quando aplicada?

 b.

Permitir, nas condições definidas, que scripts executados no navegador sob a origem http://localhost:5173 acessem respostas da API, sem substituir autenticação ou autorização.

a.

Autenticar usuários provenientes da porta 5173 e impedir chamadas realizadas por outros tipos de cliente.

 b.

Permitir, nas condições definidas, que scripts executados no navegador sob a origem http://local

host:5173
acessem
respostas
da API,
sem
substituir
autenticaç
ão ou
autorizaçã
o.

c.

Fazer com
que as
portas
5173 e
5213
sejam
considera
das a
mesma
origem
pelo
navegador
.

d.

Criar um proxy
interno
que
encaminh
a ao
ASP.NET
Core
todas as
requisiçõe
s
recebidas
pelo
React.

e.

Impedir que
clientes
que não
sejam
navegador
es
invoquem

os
métodos
autorizado
s pela
API.

Respos
ta:
B

Coment
ário
:
Add
Cor
s
regi
stra
a
polí
tica
e
Use
Cor
s a
apli
ca
ao
pip
elin
e.
Esq
ue
ma,
hos
t e
port
a
co
mp
õe
m a
orig
em.
CO
RS
con
trol
a,

nos
nav
ega
dor
es,
o
ace
sso
entr
e
orig
ens
,
ma
s
não
aut
enti
ca
o
usu
ário
ne
m
sub
stit
ui
as
regr
as
de
aut
oriz
açã
o.

Pergunta 2

0,25 em 0,25 pontos

O Vite está configurado assim:

```
const backendUrl = 'http://localhost:5213';

export default defineConfig({
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: backendUrl,
        changeOrigin: true,
        secure: false
      }
    }
  }
});
```

No React:

```
fetch('/api/produtos?page=1');
```

Qual descrição representa corretamente o fluxo durante o desenvolvimento?

F



e.

O navegador solicita
/api/... ao servidor
de
desenvolvimento
do Vite, e o proxy
encaminha a
requisição ao
backend
configurado.

f

a.

fetch identifica automaticamente a porta do ASP.NET Core e ignora o servidor Vite.

b.

A chamada chega primeiro ao ASP.NET Core, que a devolve ao Vite quando não encontra o endpoint.

c.

O proxy altera apenas a URL exibida no navegador, sem encaminhar a requisição ao backend.

d.

O Vite cria cópias dos endpoints ASP.NET Core e executa a lógica da API dentro do processo do frontend.



e.

O navegador solicita /api/... ao servidor de desenvolvimento do Vite, e o proxy encaminha a requisição ao backend configurado.

Resposta: E

Comentário: A

URL
relativa
utiliza a
origem do
frontend.
O proxy
configurad
o no
servidor
de
desenvolvi
mento do
Vite
encaminh
a as
requisiçõe
s com o
prefixo
/api ao
endereço
definido
como
destino.

Pergunta 3

0,25 em 0,25 pontos

Considere o componente React:

```
const [products, setProducts] = useState([]);
const [page, setPage] = useState(1);
const [category, setCategory] = useState("");
const [loading, setLoading] = useState(false);
const [error, setError] = useState(null);

useEffect(() => {
  setLoading(true);
  setError(null);

  listarProdutos({ page, category })
    .then(setProducts)
    .catch(setError)
    .finally(() => setLoading(false));
}, [page, category]);
```

Desconsidere eventuais execuções adicionais realizadas pelo React em modo de desenvolvimento para detectar problemas nos efeitos.

Depois da execução inicial, quais alterações fazem esse efeito ser executado novamente em razão do array de dependências?

Resposta:



d. Alterações em page ou category.

Resposta:

a.

p
o
s
t
a
s
:

Alterações em products ou loading.

b.

Alterações em error ou products.

c.

Qualquer alteração em um estado declarado pelo componente.

✔ d.

Alterações em page ou category.

e.

Apenas alterações em page, porque category contém uma string.

(Resposta: D

Comentário: page e category são as dependências declaradas. Quando um desses valores muda entre renderizações, o React executa novamente o efeito.

Pergunta 4

0,25 em 0,25 pontos

Uma tela de listagem precisa distinguir quatro situações:

1. requisição em andamento;
2. requisição concluída com registros;
3. requisição concluída sem registros;
4. requisição que falhou.

Qual estratégia de estado representa melhor essa necessidade?

R

☒ a.

Manter estados explícitos para carregamento e erro, tratar os dados em caso de sucesso e exibir “nenhum resultado” somente quando não houver carregamento, erro ou itens.

R

☒ a.

Manter estados explícitos para carregamento e erro, tratar os dados em caso de sucesso e exibir “nenhum resultado”

somente quando não houver carregamento, erro ou itens.

b.

Manter apenas products e interpretar uma lista vazia conforme o tempo transcorrido desde o início da requisição.

c.

Mostrar a mesma mensagem para os estados 1, 3 e 4 e renderizar os dados somente no estado 2.

d.

Interpretar uma resposta bem-sucedida com coleção vazia como falha HTTP, pois sucesso pressupõe pelo menos um registro.

e.

Definir loading como false antes de iniciar a requisição para evitar um estado intermediário na interface.

Resposta: A

Comentário: Uma

coleção vazia não permite distinguir, sozinha, carregamento, falha e ausência legítima de resultados.

Estados separados tornam cada situação identificável.

Pergunta 5

0,25 em 0,25 pontos

Um frontend React possui:

```
<BrowserRouter>
  <Routes>
    <Route path="/" element={<ProductListPage />} />
    <Route
      path="/products/:id"
      element={<ProductDetailsPage />}
    />
  </Routes>
</BrowserRouter>
```

A página de detalhes contém:

```
const { id } = useParams();

useEffect(() => {
  const productId = Number(id);

  if (!Number.isInteger(productId) || productId
    <= 0) {
    setError('Identificador inválido');
    return;
  }

  obterProduto(productId)
    .then(setProduct)
    .catch(setError);
}, [id]);
```

Qual alternativa descreve corretamente a divisão de responsabilidades?

 c.

O roteador do frontend escolhe o componente conforme a URL; a página obtém o parâmetro e pode fazer uma requisição independente ao endpoint da API.

a.

/products/:id é o endpoint da API, portanto o ASP.NET Core valida o parâmetro antes de o componente React ser selecionado.

b.

BrowserRouter substitui o roteamento do ASP.NET Core e passa a atender também às URLs do backend.

 c.

O roteador do frontend escolhe o componente

conforme a URL; a página obtém o parâmetro e pode fazer uma requisição independente ao endpoint da API.

d.

obterProduto() precisa alterar a URL do navegador para /api/... antes de realizar a requisição.

e.

useParams() converte automaticamente parâmetros numéricos para number, tornando a conversão explícita desnecessária.

Resposta: C

Comentário: A

rota da SPA determina qual componente é exibido. A comunicação com a API é uma operação HTTP separada, realizada depois que o componente obtém o

parâmetro
da URL.

Pergunta 6

0,25 em 0,25 pontos

Um usuário efetuou login corretamente. Sua identidade foi reconhecida e os dados enviados por ele são válidos. Entretanto, esse usuário não possui permissão para alterar determinado recurso.

Qual mecanismo deve impedir a operação?

Respos
ta
Sel
eci
on
ad
a:



e.

Autorização, porque a identidade já foi confirmada e é necessário verificar se ela possui acesso à operação.

Respos
tas:

a.

Autenticação, porque qualquer acesso negado significa que a identidade não foi confirmada.

b.

Validação, porque permissões são tratadas como restrições dos dados enviados.

c.

Sessão, porque somente dados armazenados nela podem estabelecer permissões.

d.

Model binding, porque ele determina quais funcionalidades podem ser executadas pelo usuário.

 e.

Autorização, porque a identidade já foi confirmada e é necessário verificar se ela possui acesso à operação.

Com
e
n
t
á
ri
o
d
a
r
e
s
p
o
s
t
a
:

Resposta: E

Comentário: Autenticação estabelece a identidade. Autorização decide quais operações essa identidade pode realizar. A validação dos dados trata outra responsabilidade.

Pergunta 7

0,25 em 0,25 pontos

Considere dois cenários:

Aplicação A: páginas e formulários utilizados diretamente em um navegador.

Aplicação B: API consumida por um aplicativo móvel e por outros clientes independentes.

Qual alternativa descreve corretamente um uso comum de cookies de autenticação e tokens JWT?

R

 c.

Em aplicações web, o navegador pode enviar automaticamente o cookie de autenticação quando suas regras permitem. Em APIs que usam JWT como bearer token, o cliente normalmente envia o token no cabeçalho Authorization.

R

a.

O navegador envia automaticamente qualquer JWT armazenado pelo cliente, enquanto cookies precisam ser inseridos manualmente no cabeçalho Authorization.

b.

Cookies de autenticação são utilizados somente para sessões anônimas, enquanto JWT é necessário para qualquer usuário identificado.

 c.

Em aplicações web, o navegador pode enviar automaticamente o cookie de autenticação quando suas regras permitem. Em

APIs que usam JWT como bearer token, o cliente normalmente envia o token no cabeçalho Authorization.

d.

Cookies podem armazenar somente roles, enquanto tokens JWT podem transportar apenas claims que não identificam o usuário.

e.

Depois que um cookie ou JWT é emitido, o servidor não precisa validar a credencial nas requisições posteriores.

Resposta: C

Comentário: cookies integram-se ao mecanismo de envio de credenciais do navegador. Quando um JWT é utilizado como bearer token, o cliente normalmente controla seu envio no cabeçalho HTTP. Nos dois casos, o servidor precisa validar a credencial.

Pergunta 8

0,25 em 0,25 pontos

Uma aplicação configurou o pipeline assim:

```
app.UseAuthorization();  
app.UseAuthentication();  
  
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

Uma ação contém:

```
[Authorize(Roles = "Editor,Administrador")]
```

Qual alteração é necessária?

f  a.
Colocar
 UseAuthentication()
) antes de
 UseAuthorization()
para estabelecer a
identidade antes
da avaliação das
regras de acesso.

f  a.
Colocar
 UseAuthentication()
) antes de
 UseAuthorization()
para estabelecer a
identidade antes

da avaliação das regras de acesso.

b.

Remover

UseAuthorization()
, pois [Authorize]
executa toda a
autorização sem
middleware.

c.

Remover [Authorize],
pois autorização
baseada em roles
é exclusiva de
Minimal APIs.

d.

Mapear os

controladores
antes dos dois
middlewares para
que a ação seja
descoberta antes
da autenticação.

e.

Executar

UseAuthentication(
) duas vezes, uma
para cada role
definida.

Resposta: A

Comentário: A

autorizaçã
o precisa
avaliar
uma
identidade
previamen
te
autenticad
a. Por
isso, o

middleware
e de
autenticaç
ão deve
ser
executado
antes do
middleware
e de
autorizaçã
o.

Pergunta 9

0,25 em 0,25 pontos

Uma organização possui a role:

Analista

e a claim:

Permissao = AprovarCadastro

Todos os analistas podem acessar a área de análise, mas somente alguns podem executar a operação específica de aprovação.

Qual desenho é mais adequado?

I

 d.

Utilizar a role Analista
para o acesso
geral e a claim de
permissão para
representar a
capacidade
específica de
aprovar cadastros.

I

a.

Utilizar a claim para
representar o
acesso geral e a
role para a
operação
específica, pois
roles são
necessariamente
mais granulares.

b.

Utilizar somente claims
porque a presença
de uma claim torna
roles
incompatíveis com
a identidade.

c.

Utilizar somente roles
porque claims
servem
exclusivamente
para exibir
informações do
perfil.

✓ d.

Utilizar a role Analista para o acesso geral e a claim de permissão para representar a capacidade específica de aprovar cadastros.

e.

Enviar role e claim em campos ocultos do formulário para que o próprio usuário informe suas permissões ao servidor.

Resposta: D

Comentário:

Uma role pode representar um agrupamento amplo de acesso, enquanto uma claim pode registrar uma característica ou permissão específica da identidade.

0,25 em 0,25 pontos

Uma SPA utiliza autenticação baseada em cookie. Para consultar o usuário atual, executa:

```
fetch('/api/auth/me', {  
  credentials: 'include'  
});
```

O servidor protege os recursos que exigem autenticação. No frontend, existe também:

```
function ProtectedRoute() {  
  const { authenticated } = useAuth();  
  
  if (!authenticated)  
    return <Navigate to="/entrar" replace  
    />;  
  
  return <Outlet />;  
}
```

Qual avaliação está correta?

✔ b.

O estado utilizado por ProtectedRoute melhora o fluxo de navegação, mas o servidor continua responsável por proteger os recursos. A opção credentials controla o envio de credenciais nas requisições realizadas por fetch.

a.

ProtectedRoute impede por si só o acesso aos recursos protegidos, portanto a API não precisa aplicar autorização.

✔ b.

O estado utilizado por ProtectedRoute melhora o fluxo de navegação, mas o servidor continua responsável por proteger os recursos. A opção credentials controla o envio de credenciais nas requisições realizadas por fetch.

c.

credentials: 'include'
valida o cookie no
navegador antes de
a requisição ser
enviada,
dispensando
validação no
servidor.

d.

A SPA deve copiar o
conteúdo do cookie
para localStorage
para decidir se uma
rota pode ser
acessada.

e.

Quando existe uma rota
protegida no React,
requisições diretas
aos endpoints
correspondentes
também passam
pelo
ProtectedRoute.

Resposta: B

Comentário: A

proteção
no
frontend
controla a
experiênci
a de
navegaçã
o, não o
acesso
efetivo ao
recurso.
Como a
API pode
ser
chamada
diretament
e, a

proteção
precisa
existir
também
no
servidor.

Questionário IV

Pergunta 1

0,25 em 0,25 pontos

Uma aplicação possui três necessidades de observabilidade:

1. registrar detalhes estruturados de uma falha de autenticação;
2. contabilizar quantas falhas ocorreram ao longo do tempo;
3. correlacionar as etapas executadas durante uma mesma operação.

Qual conjunto associa corretamente cada necessidade ao mecanismo correspondente?

Resposta

Selecione a:



d.

ILogger, Counter<long> e
ActivitySource.

Respostas:

a.

Health checks, ILogger e output
cache.

b.

ActivitySource, ILogger e
Counter<long>.

c.

Counter<long>, ActivitySource e
ILogger.



d.

ILogger, Counter<long> e
ActivitySource.

e.

ILogger, health checks e
Counter<long>.

Com
e
n
t
â
r
i
o
d
a
r
e
s
p
o
s
t
a
:

Resposta: D

Comentário: logs registram eventos e
seus contextos. Métricas
representam valores agregáveis,
como contagens. O rastreamento
permite correlacionar atividades
pertencentes a uma mesma
operação.

Pergunta 2

0,25 em 0,25 pontos

Uma aplicação registra uma verificação de banco de dados:

```
builder.Services
    .AddHealthChecks()
    .AddCheck<DatabaseHealthCheck>("database");
```

e expõe:

```
app.MapHealthChecks(
    "/health/live",
    new HealthCheckOptions
    {
        Predicate = _ => false
    });

app.MapHealthChecks("/health/ready");
```

O processo continua executando, mas o banco de dados fica indisponível. Qual resultado é coerente com essa configuração?

 c.

/health/live pode
continuar saudável
porque exclui as
verificações
registradas,
enquanto
/health/ready pode
indicar falha em
razão da
indisponibilidade
do banco.

a.

Ambos os endpoints falham porque toda verificação registrada é executada em qualquer `MapHealthChecks()`.

b.

`/health/live` falha e `/health/ready` permanece saudável porque `readiness` não pode consultar dependências externas.

✔ c.

`/health/live` pode continuar saudável porque exclui as verificações registradas, enquanto `/health/ready` pode indicar falha em razão da indisponibilidade do banco.

d.

Ambos permanecem saudáveis porque `health checks` não devem representar disponibilidade de banco.

e.

`/health/ready` somente pode falhar quando o próprio processo ASP.NET Core for encerrado.

Resposta: C

Comentário:

Os dois endpoints podem avaliar condições diferentes. O predicado usado em /health/live exclui os checks registrado s, enquanto /health/ready executa a verificação do banco.

Pergunta 3

0,25 em 0,25 pontos

Uma aplicação consulta um serviço externo. A política deve:

- repetir a chamada até três vezes diante de falhas transitórias;
- após falhas repetidas, interromper temporariamente novas chamadas à dependência;
- durante esse período, permitir que a aplicação adote uma resposta alternativa.

Quais padrões correspondem aos dois primeiros comportamentos?

Resposta Seleccionada:



e.
Retry e circuit breaker.

Respostas:

- a.
Rate limiting e output caching.
- b.
Duas políticas de retry com intervalos diferentes.
- c.
Circuit breaker e autenticação.
- d.
Output caching e readiness check.



e.
Retry e circuit breaker.

Resposta: E

Comentário: retry repete operações que podem se recuperar de falhas transitórias. Circuit breaker interrompe temporariamente novas tentativas quando a

dependência
apresenta falhas
repetidas.

Pergunta 4

0,25 em 0,25 pontos

Uma aplicação possui:

GET /api/catalogo	→ catálogo público
GET /api/catalogo/{id}	→ detalhe público
POST /api/auth/login	→ autenticação
GET /api/perfil	→ dados personalizados do usuário

Qual combinação de output cache e rate limiting é mais coerente?

R

✓ a.

Considerar output cache para respostas públicas adequadas do catálogo, aplicar limite mais restritivo ao login e não tratar o perfil autenticado como resposta de cache compartilhado.

R

✓ a.

Considerar output cache para respostas públicas adequadas do catálogo, aplicar limite mais restritivo ao login e não tratar o perfil autenticado como resposta de cache compartilhado.

b.

Utilizar cache compartilhado em /api/perfil e /api/auth/login e dispensar limites no login.

c.

Armazenar em cache todos os GET, inclusive /api/perfil, porque o verbo HTTP é suficiente para determinar se o compartilhamento é seguro.

d.

Aplicar rate limiting somente aos endpoints públicos porque a

validação da senha já impede
abuso do login.

e.

Aplicar exatamente a mesma
política de cache e de limite a
todos os endpoints para
manter comportamento
uniforme.

Resposta: A

Comentário: cache e
limitação resolvem
problemas
diferentes.
Respostas públicas
podem ser
candidatas a
cache, enquanto
endpoints sensíveis
a abuso, como
login, podem exigir
limites mais
restritivos.

Pergunta 5

0,25 em 0,25 pontos

Uma aplicação registra políticas:

```
builder.Services.AddRateLimiter(...);  
builder.Services.AddOutputCache(...);
```

e configura:

```
app.MapGet("/api/dados", ObterDados)  
    .RequireRateLimiting("public-api")  
    .CacheOutput("public-cache");
```

Entretanto, não existem as chamadas:

```
app.UseRateLimiter();  
app.UseOutputCache();
```

Qual análise está correta?



b.

Os serviços e as políticas foram registrados e associados ao endpoint, mas os respectivos middlewares ainda precisam integrar o pipeline.

a.

As políticas funcionam

normalmente porque os metadados do endpoint substituem os middlewares.



b.

Os serviços e as políticas foram registrados e associados ao endpoint, mas os respectivos middlewares ainda precisam integrar o pipeline.

c.

O output cache funciona pelo endpoint, mas o rate limiting exige middleware.

d.

O rate limiting funciona pelo endpoint, mas o output

cache
exige
middleware
e.

e.

A aplicação
necessari
amente
falha na
compilaçã
o porque
RequireR
ateLimitin
g() e
CacheOut
put() só
existem
depois
das
chamadas
Use...().

Respos
ta:
B

Coment
ário
:
Re
gist
rar
ser
viç
os,
ass
oci
ar
polí
tica
s a
end
poi
nts
e
ins
erir
os

middlewares no pipeline são etapas diferentes. A ausência dos middlewares impede a aplicação dos componentes esperados.

Pergunta 6

0,25 em 0,25 pontos

Uma aplicação possui três tipos de configuração:

- nome público da aplicação;
- URL de um serviço usada especificamente em Staging;
- chave secreta de acesso a uma API externa.

A equipe propõe:

appsettings.json	→ nome público
appsettings.Staging.json	→ URL de Staging
User Secrets	→ chave durante desenvolvimento
variável de ambiente	→ chave real em produção

Qual avaliação é a mais adequada?

✔ c.

A separação é apropriada: valores não sensíveis podem ser organizados por ambiente, enquanto segredos devem permanecer fora dos arquivos versionados.

a.

Todas as configurações deveriam ficar em appsettings.json,

pois arquivos
específicos de
ambiente e
variáveis servem
apenas ao sistema
operacional.

b.

A chave deveria ficar
em
appsettings.Produc
tion.json, pois um
arquivo específico
de produção
constitui
armazenamento
seguro de
segredos.

✔ c.

A separação é
apropriada: valores
não sensíveis
podem ser
organizados por
ambiente,
enquanto segredos
devem permanecer
fora dos arquivos
versionados.

d.

Cada ambiente exige
nova compilação
porque variáveis
de ambiente não
participam do
sistema de
configuração do
ASP.NET Core.

e.

User Secrets é o
mecanismo
indicado para
distribuir segredos
às máquinas de
produção.

Resposta: C

Comentário:

Valores
não
sensíveis
podem ser
definidos
em
arquivos
de
configuraç
ão gerais
ou
específico
s de
ambiente.
Segredos
exigem
mecanism
os que
evitem sua
inclusão
no
código-font
e e em
arquivos
versionado
s

Pergunta 7

0,25 em 0,25 pontos

O arquivo de configuração contém:

```
{
  "Sistema": {
    "ServicoExterno": {
      "ApiKey": ""
    }
  }
}
```

Qual nome de variável de ambiente representa essa chave hierárquica de forma compatível entre plataformas no sistema de configuração do ASP.NET Core?

Resposta
Selecionada:
a:



b.

Sistema__ServicoExterno__ApiKey

Respostas:

a.

Sistema.ServicoExterno.ApiKey



b.

Sistema__ServicoExterno__ApiKey

c.

Sistema::ServicoExterno::ApiKey

d.

Sistema_ServicoExterno_ApiKey

e.

Sistema/ServicoExterno/ApiKey

C Resposta: B

Comentário: O
provedor de
variáveis de
ambiente
interpreta __
como separador
hierárquico
equivalente a : na
configuração.

Pergunta 8

0,25 em 0,25 pontos

Considere o Dockerfile:

```
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS base
WORKDIR /app
```

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
```

```
COPY ["Aplicacao.csproj", "."]
RUN dotnet restore "Aplicacao.csproj"
COPY . .
RUN dotnet publish "Aplicacao.csproj" \
    -c Release -o /app/publish
```

```
FROM base AS final
WORKDIR /app
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "Aplicacao.dll"]
```

Qual é uma finalidade importante dessa construção em múltiplos estágios?

R



d.

Separar as ferramentas necessárias ao build das dependências necessárias à execução, copiando para a imagem final os artefatos publicados.

R

a.

Manter o SDK e o código-fonte na imagem final para recompilar a aplicação sempre que o contêiner iniciar.

b.

Incorporar automaticamente as variáveis de ambiente de produção ao conteúdo permanente da imagem.

c.

Eliminar a etapa de publicação, pois a imagem aspnet executa diretamente arquivos .cs.



d.

Separar as ferramentas necessárias ao build das dependências necessárias à execução, copiando para a imagem final os artefatos publicados.

e.

Executar simultaneamente uma instância de desenvolvimento no estágio build e outra de produção no estágio final.

Resposta: D

Comentário: O SDK é utilizado para restaurar, compilar e publicar a aplicação. A imagem final pode conter apenas o runtime e os artefatos necessários à execução.

Pergunta 9

0,25 em 0,25 pontos

Uma solução full-stack pode utilizar uma destas estratégias:

- I. O ASP.NET Core serve a API e os arquivos compilados do frontend.
- II. Frontend e API são implantados separadamente e possuem ciclos de implantação independentes.

Qual comparação está correta?

R ☒ a.

A estratégia I pode facilitar uma implantação conjunta e o uso da mesma origem. A estratégia II permite maior independência de implantação, mas exige coordenação entre os contratos utilizados pelo frontend e pela API.

R ☒ a.

A estratégia I pode facilitar uma implantação conjunta e o uso da mesma origem. A estratégia II permite maior independência de implantação, mas exige coordenação entre os contratos utilizados pelo frontend e pela API.

b.

A estratégia I exige que frontend e API utilizem domínios diferentes. A estratégia II exige

que ambos utilizem o mesmo domínio.

c.

A estratégia II elimina a necessidade de compatibilidade entre frontend e API porque cada aplicação possui seu próprio pipeline.

d.

A estratégia I impede que arquivos estáticos sejam armazenados em cache, enquanto a estratégia II permite cache apenas no frontend.

e.

Na estratégia II, alterações incompatíveis na API não afetam versões anteriores do frontend porque os dois sistemas são implantados separadamente.

Resposta: A

Comentário: Separar as implantações aumenta a autonomia operacional, mas não elimina a dependência funcional entre consumidor e API. Uma mudança incompatível de contrato ainda pode afetar o frontend.

Pergunta 10

0,25 em 0,25 pontos

Uma aplicação deve servir uma SPA apenas sob o prefixo /app, sem interferir nas rotas /Pets, /api/produtos, /health/live e /health/ready. A configuração é:

```
var indexPath = Path.Combine(
    app.Environment.WebRootPath,
    "app",
    "index.html");

if (File.Exists(indexPath))
{
    app.MapFallbackToFile(
        "/app/{*path:nonfile}",
        "app/index.html");
}
```

Qual é o efeito pretendido?

R



e.

Quando o arquivo existir, o fallback ficará restrito ao prefixo /app, permitindo que as rotas de MVC, API e health checks fora desse prefixo continuem sendo tratadas por seus endpoints.

R

a.

Se index.html não existir, qualquer requisição para /app será redirecionada automaticamente para /Pets.

b.

File.Exists() verificará o arquivo apenas durante a compilação, de modo que o fallback será registrado mesmo quando a condição for falsa.

c.

O fallback fará /api/produtos devolver index.html sempre que uma consulta válida não encontrar registros.

d.

MapFallbackToFile() ignorará o template informado e

capturará qualquer URL desconhecida da aplicação.



e.

Quando o arquivo existir, o fallback ficará restrito ao prefixo /app, permitindo que as rotas de MVC, API e health checks fora desse prefixo continuem sendo tratadas por seus endpoints.

Resposta: E

Comentário: Restringir o padrão do fallback evita que rotas do servidor sejam confundidas com rotas internas da SPA. A condição também impede o registro do fallback quando o arquivo do frontend não existe.